

Live Game Operations Rooms

Approach

3 hour goal

- 1h Thinking
- 1h Coding
- 1h Testing/Deliverables

3 hour goal

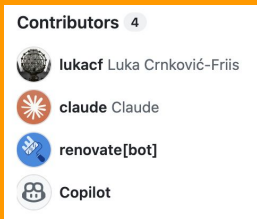
1h Coding hinges on plug n play tools (no bugs, well documented)

- Assignment requires <https://github.com/lukacf/meerkat>

CAUTION

Flagged in first
10 minutes

meerkat has a **single** (human) **contributor** and is **largely vibe** coded



- Total commits: 4,265

So the repo is about 7 months old
~1.6M hand-written code lines

→ In prod, I would recommend frameworks with a community and/or long term guarantees

> what are open source and long term alternatives to using meerkat + meerkat-mob?

If the identity-first multi-agent team concept (Meerkat's core differentiator) is what you want to preserve, these are the closest matches:

- **CrewAI** — the most direct conceptual analog to Meerkat's "mobs": role-based, reusable agent teams with defined goals; the most approachable option for building agent teams, open source with an enterprise tier.[langfuse+1](#)
- **AWS Multi-Agent Orchestrator** — Bedrock-native routing between multiple agents, free SDK, good fit if standardizing on [AWS.ayautomate](#)
- **AgentScope** — distributed, observable multi-agent framework from Alibaba Cloud, built for high-throughput scenarios.[ayautomate](#)
- **Microsoft Conductor** — described as "the most credible long-term bet" among newer multi-agent CLI orchestrators, MIT-licensed, using YAML-defined workflows with a web dashboard, built on GitHub Copilot SDK and Claude.

3 hour goal

1h Coding hinges on plug n play tools (no bugs, well documented)

1. Fallback to Figma/Excalidraw - not acceptable vs AI-skeptic stakeholders
2. Replace meerkat - break meerkat requirement
3. Fill the gaps in meerkat - break time requirement
 - ◆ Small extra time > completely different library
 - ◆ Stick with existing ecosystem
 - ◆ Curiosity
 - ◆ One extra to discuss later

Thinking / Coding split

- Demo first
 - ~ 1 full page of requirements
 - Emphasis (“Core requirement”, “Runnable implementation”)
 - Live usage informs UI and architecture decisions
- Architecture second
 - Requirement have 2 lines at the bottom
 - Interview offers opportunity to discuss more
 - Findings from research show simple architectures
 - HolmesGPT uses controller/worker setup
 - Domain specialization (+memory, source-ids, post-mortem, handoff, etc.)
hard to devise from project description

Assignment framing

Assignment is specifics-first

Open Context	Specific Implementation
● “Support an SRE team” ● “No correct answer” ● “Purposeful design” ● “Minimal UI”	● 2+ agents minimum ● Different responsibilities/context ● Real mob collaboration ● Stable source ids ● Handle late evidence ● Preserve earlier evidence ● Account for cost ● Show hypothesis confidence ● Show impact ● Show next action ● Show Agent status ● Produce handoff ● No chain-of-thought ● Show supporting/conflicting evidence

Is this a mangled real-world problem description?

Is there a whole hidden question about RAG for incidents?

Candidate enjoyability

Open Implementation	Specific Context
• “Show that team some sort of demo”	• “The incident team responds to incidents with Architecture X and find Y and Z difficult and boring. They have ecosystem X in place, they are accountable for Y, and don’t trust AI.” • Ecosystem uses GTP-5.6 Sol for all agents, here is a key valid for 14 days: <code>OPENAI_API_KEY=sk-...</code>

Time breakdown

Honest breakdown - User time 2.4h

RCA control
1.2h

Web UI
0.9h

Web UI — user 53.8m (0.9h) · wall 109.7m (1.8h)

- Scaffold from mockup (UI_v1.png) — user 16.3m · wall 43.0m
 - Build approach + first cut — user 11.6m · wall 28.6m
 - "proceed" → first render — user 4.7m · wall 14.4m
- Panels & correctness — user 20.9m · wall 32.5m
 - Nail behavior: show fact-answered tests — user 14.9m · wall 21.2m
 - Keep working on UI behavior — user 4.2m · wall 4.6m
 - Test status chips (YES/NO/Pending) + falsifier links — user 1.8m · wall 6.7m
- Operator-chat UX — user 14.1m · wall 29.7m
 - Live status msgs ("creating hypotheses... done") — user 12.3m · wall 21.7m
 - Lock chat while a run is in progress — user 1.8m · wall 8.0m
- SUB AGENT gag button (rickroll) — user 1.6m · wall 3.6m
 - Add the blue button — user 1.4m · wall 3.0m
 - Gate it on a pending test being selected — user 0.2m · wall 0.6m
- Redeploy & verify — user ~0.9m · wall ~0.9m

Prompts — user 10.9m (0.18h) · wall 15.6m

- Tighten a gen prompt + bump model to gpt-4.1 — user 2.3m · wall 4.2m [session 5]
- "switch to 4.1 and tighten the prompt as well"
- Regenerate / review both generation prompts — user 5.6m · wall 5.7m [session 6]
- "give me both prompts again" (hypothesis-gen + test-gen)
- Operator voice — informal, French-interjecting dev — user 3.0m · wall 5.7m [session 6]
- "...the style should be informal, like an old school bearded developer, who often interject in French"

Seed — user 5.6m (0.09h) · wall 10.9m

- Wire the incident.json / late-arrival.json schema — user 5.6m · wall 10.9m [session 6]
- "I added incident.json and late-arrival.json to ace/ that show the correct schema... make sure our system can handle that input schema"

RCA protocol / controller — user 71m (1.18h) · wall ~136m

- A. Protocol design (up-front spec) — user 14.6m · wall 32.4m
 - "build an RCA system..." initial spec — user 1.1m · wall 4.3m
 - blocks—parent edges + falsification rule — user 9.8m · wall 11.3m
 - "implement this plan; roster filled on compose up" — user 3.7m · wall 16.8m (Claude then built it)
 - B. Graph-correctness enforcement — user ~18m · wall ~34m
 - "single hypothesis has 2 tests" — user 7.0m · wall 13.1m
 - "test_runner validating a hyp is unacceptable" — user 3.4m · wall 10.0m
 - "diagnosis tests never written if answer in facts?" — user 2.7m · wall 3.2m
 - "generate_tests gated / 1 test at a time" — user 2.8m · wall ~5m
 - "when all hyps fail, regenerate new ones" — user 2.1m · wall 4.6m
 - C. Drive-model pivot (turn-based → controller-driven) — user 16.3m · wall 37.4m
 - "Go for Python + switch back to gpt-4o-mini" — user 2.6m · wall 9.0m
 - "querying each agent once is not acceptable" — user 4.0m · wall 10.8m
 - controller log paste (RPC ready, watching coord.) — user 5.4m · wall 9.1m
 - "do both" — user 2.2m · wall 3.2m
 - (s6) "redeploy, try controller precise link" — user 2.0m · wall 5.3m
 - D. Graph-state VERIFICATION (deploy → inspect → confirm) — user 22.1m · wall 31.0m
- short messages; the minutes are spent DRIVING the live system, not typing:
- "process finished, does the graph have expected state?" — user 4.9m · wall 7.3m
 - "it finished, does the graph state look correct?" — user 5.0m · wall 6.1m
 - "I added this fact, is the graph correct?" — user 5.3m · wall 6.1m ← 8-word msg
 - "do it + default facts to incident + cosmetic nodes" — user 3.1m · wall 6.3m
 - "check it" — user 3.8m · wall 4.4m
 - "is the graph state sensible now?" — user 0.0m · wall 0.9m

Prompts 0.2h

Seed 0.1h

Complete Honest breakdown - Wall clock time 7.1h



Multitask time!

Completely Honest breakdown - meerkat gaps 13h

User build
2.4h

Claude build
2.2h

User library tax
3.1h

Claude library tax
2.8h

Compile time
2.5h

	FAILURE MODE	DOC VERDICT
A <code>turn_driven</code> member answers only its FIRST <code>console/send</code> per session; 2nd+ return an empty frame in ~1ms — fix is <code>respawn_member</code> before every ask	SILENT	Undoc
The <code>WorkGraph</code> grant is auto-issued by <code>persistent_state(dir)</code> — docs imply <i>you</i> must issue it	MISLEADING	Misleading
OpenAI 400s the top-level <code>not</code> in <code>workgraph_claim</code> ; the <code>openai_compatible</code> adapter won't lower it (also hit as a <code>llama.cpp</code> boolean-schema shim)	SILENT	Undoc · defect
No <code>tracing</code> subscriber ⇒ every meerkat warn/error silently dropped — see the box below	SILENT	Undoc
Non-orchestrator profiles never auto-spawn — only <code>coordinator</code> reconciles on boot; need <code>ensure_member</code>	SILENT	Vague
Console binaries bind <code>127.0.0.1:0</code> only — no flag/env/config re-binds it; forced compiling a library host (whole Rust build saga downstream)	HARD-FAIL	Partial
<code>build_runtime_decision_state</code> validators (JWKS keys, audience) crash startup	HARD-FAIL	Partial
A member with <code>tools.comms=false</code> is hard-rejected — there is no tool-free member	HARD-FAIL	Undoc
<code>mobkit/reset_all</code> takes the whole stack unreachable (HTTP 000) — soft-reset epoch instead	TRAP	Undoc
The <code>labels</code> filter is match-ALL (AND) — a multi-label query returns <code>[]</code>	SILENT	Undoc
Terminal items are hidden unless you pass <code>include_terminal:true</code>	SILENT	Vague
<code>workgraph/evidence/add</code> needs its own inner <code>id</code> on the evidence object or it fails	TRAP	Undoc

Retrospective

If assignment drops at lunch, presenting the next morning?

- Back to [slide 6](#)
 - Meerkat risk was flagged very early
 - Manage expectations
 - Involve stakeholders in path forward
 - Reach out to repo creator to flag landmines early

Image artefacts

Similar incidents

Xxx
yyy

Hypotheses

H1 New deployment
~~H2 DoS~~

Tests

T0 CPU above 80%?
T1 RAM above 99%?

Operator chat

SEND

Hypotheses (supporting | conflicting facts)

(open) H001 (3 | 0) New deployment
(failed) H002 (1 | 2) DoS attack

Tests (related hypo) (outcome)

T002 (H002) Is CPU above 80%?
T001 (H001) Is RAM above 99%? (No)

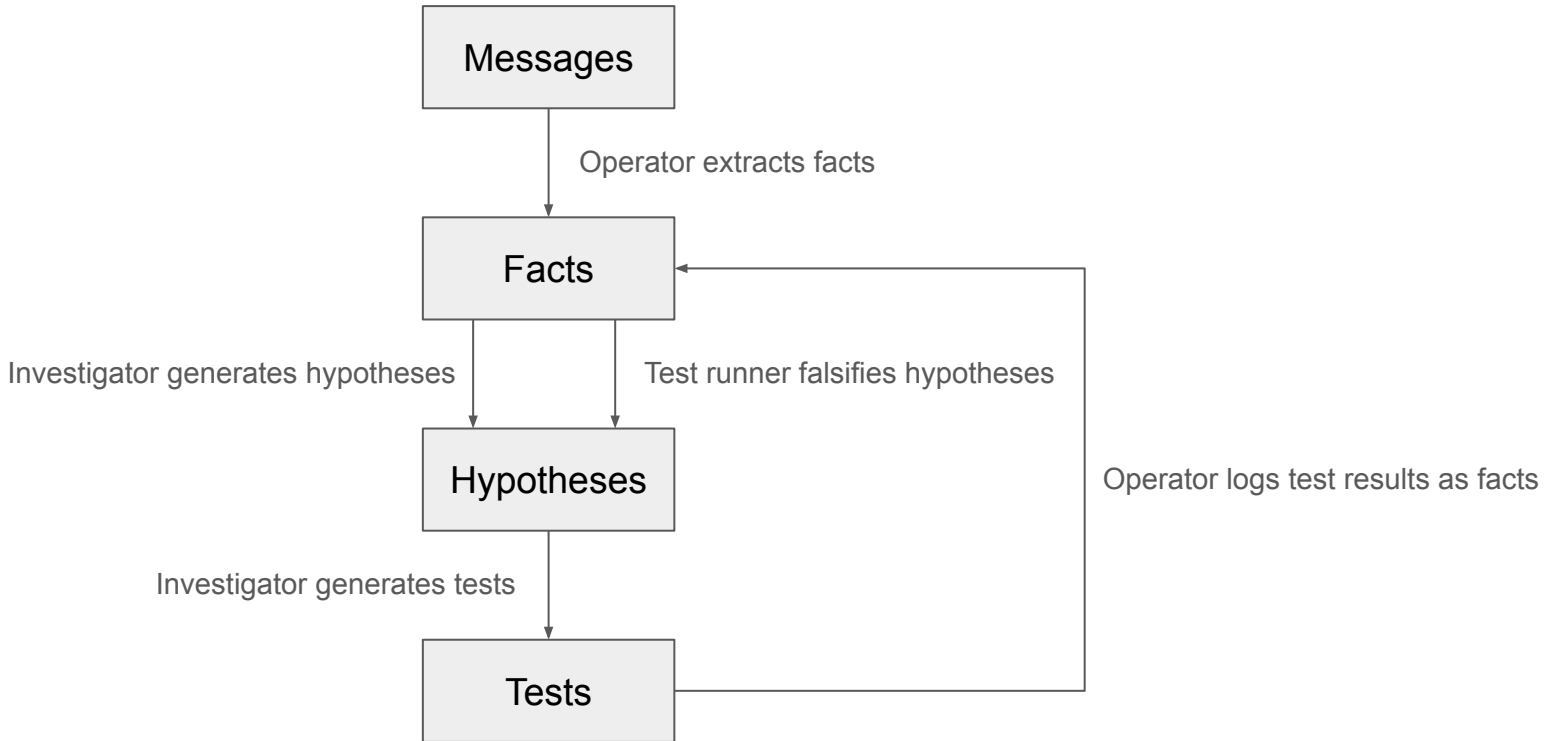
YES

NO

Operator chat

User: New Incident xxx,
facts xxx, yyy
Operator: Updated
workgraph

SEND



Notes on deliverables

- “runnable implementation” smuggles overhead and decisions
 - What machine I/you have -> Design guessing
 - Repo is also required, which presumably must produce identical runtime -> CI/CD
 -